

VOLT — Whitepaper

A price arcade whose results settle on records the house does not own.

Gabit · Giwa Sepolia (chainId 91342) · v1, 2026-07

Contents

1. The problem with "provably fair"
 2. Design principle
 3. Tap Trading
 4. Measuring volatility
 5. VOLT: a declared market
 6. Death Fun
 7. Custody and settlement
 8. Attack surface
 9. What is verifiable, and what is still trusted
 10. Deployment status
 11. Parameters
-

1. The problem with "provably fair"

A crypto casino publishes `sha256(serverSeed)` before a round and the seed after it. Replay the seed, get the same outcome. This is presented as proof of fairness. It is not.

It proves **consistency** : the house did not change its mind after seeing your bet. It says nothing about whether the number was fair **before** it was committed. The house still generates the outcome, and the player still audits the house against the house's own record. Nothing in that loop is external.

Two further gaps usually go unmentioned:

- The odds are not invertible. A payout table is posted as fact. A player has no way to work backwards from it to the actual house edge, so "1% edge" is a marketing claim rather than a derivable quantity.
- The balance is a database row. Player funds sit in a company hot wallet. Solvency is asserted, never demonstrated.

2. Design principle

Settle on a record the house does not produce, cannot alter, and does not control the publication of.

For a price game that record already exists: the price an exchange published at a given second. It is public, immutable after the fact, and queryable by anyone, indefinitely. A player does not have to trust our log — they can ask the source.

Everything below follows from that one commitment.

3. Tap Trading

The chart is a live BTC/USD trade series. The player taps a cell on a grid; each cell is a **price band** and a **settlement second** . If the price is inside that band when the second arrives, the stake pays the cell's multiplier.

The multiplier

A cell pays the reciprocal of its probability, less the edge:

```
multiplier = (1 - HOUSE_EDGE) / P(hit)
```

with `HOUSE_EDGE = 0.07`. `P(hit)` is computed, not chosen. Over a horizon `h` seconds the terminal log-price is approximately Gaussian, so for a band `[lo, hi]` measured relative to the current price:

```
σ = price × vol × √h  
P(hit) = Φ(hi/σ) - Φ(lo/σ)
```

Three consequences fall out, and they are the product:

1. Every cell carries the same expected value. A 5× cell and a 50× cell are priced identically. There are no trap cells, and no reason to build one.
2. The edge is derivable. Since `multiplier = (1 - edge)/P`, anyone who can compute `P` can invert the published payouts and recover the exact edge. It cannot be misstated.
3. The house never touches the outcome. It quotes odds. The exchange decides.

Cells that are not offered

A multiplier is never clamped — clamping distorts the edge. Instead a cell is offered only when its fair price lands in `(1×, MAX_MULT]`, with `MAX_MULT = 50`. Cells that are too likely ($\leq 1\times$) or too unlikely ($> 50\times$) simply do not appear, so every cell that is offered pays exactly `(1 - edge)/P`.

`MAX_MULT` is load-bearing risk control rather than a display choice. Payouts are the reciprocal of a probability, so the largest multipliers sit furthest into the tail, and tail probabilities are exponentially sensitive to the volatility used to compute them. At 50 a plausible mis-estimate dents the edge; at 100 it can invert it.

Band geometry

Band height is derived from volatility too, at a reference horizon:

```
step = σ_ref × 0.2436,    σ_ref = price × vol × √28
```

so the grid holds a constant shape as the market speeds up and slows down — what changes is the price height of a row, not the ladder of multipliers.

4. Measuring volatility

This is the part that decides whether any of the above is true, and it is where the actual difficulty of the product sits.

Why the naive estimate fails

Trade prints alternate between bid and ask. Tick-by-tick realized variance reads that bounce as volatility. On a simulated BTC feed with a realistic spread the naive estimator reports **6.8× the true value**. Multipliers are close to linear in the volatility they are handed, so that error would land straight in the payouts — in the player's favour, systematically.

Two-scale estimator

Microstructure noise contributes a term inversely proportional to the sampling interval, while true variance does not:

$$V(\Delta) = \sigma^2 + c/\Delta$$

Sampling the same window on a fine grid (1 s) and a coarse grid (5 s) gives two equations, so **c** can be solved for and subtracted:

$$c = (V_{\text{fine}} - V_{\text{coarse}}) / (1/\Delta_{\text{fine}} - 1/\Delta_{\text{coarse}})$$

$$\sigma^2 = V_{\text{fine}} - c/\Delta_{\text{fine}}$$

Refusing rather than guessing

The estimator returns **nothing** rather than something wrong, and the game stops quoting when it does. It refuses when:

Condition	Meaning
fewer than 40 fine returns, or 8 coarse	not enough history yet
$\sigma^2 / V_{\text{fine}} < 0.65$	noise dominates the signal
$\text{vol} < 2 \times 10^{-5}$ or $> 8 \times 10^{-4}$	outside the guard rails

The signal-to-noise gate matters most. A wide spread does not produce a slightly wrong estimate; it produces one that is mostly spread. The correct response is to stop taking bets.

Volatility is refused, never clamped . Clamping into a range would keep the game running while quietly mispricing it — the exact failure the gate exists to prevent.

Choosing the window

Pricing takes the **larger** of a 600-second and a 90-second estimate. Volatility clusters, and the player chooses when to bet: any window that lags a spike is a window they can wait for. The larger of the two never lags.

Measured result

Monte-Carlo over the realistic-spread regimes (`npm run verify:edge`):

Volatility	Median error	Realised edge
4×10^{-5}	2.3%	6.51%
1×10^{-4}	2.1%	6.70%
2.5×10^{-4}	2.5%	6.75%

Against a 7% target. The gap is the estimator's error, and it is stated rather than rounded away: the **edge** is exact given the volatility, and a real estimate carries error.

5. VOLT: a declared market

Tap Trading also runs on VOLT, a synthetic market generated in the browser from a seed.

Its volatility is not estimated. It is **declared** on a public schedule — log-space interpolation between a calm and a storm regime on two incommensurable periods (97 s and 31 s), σ swinging roughly 7× between them. With $\sigma(t)$ known, the terminal variance over any horizon is the exact integral $\int \sigma^2 dt$, so **P(hit)** is computed rather than measured and the edge is exact: EV = 0.93 on all 288 offered cells, maximum deviation 1.11×10^{-16} (`npm run verify:synth`).

VOLT exists for three reasons. A demo has to work with no network, and exchange sockets are geo-blocked in some regions. A quiet real market pauses betting — correct, but boring. And a market whose σ is known is the only way to demonstrate the pricing engine without the estimator's error in the way.

The interesting property is not that it is more volatile. The grid is **scale-invariant** in σ : doubling a **constant** volatility widens every band by the same factor and leaves the multiplier ladder unchanged. The drama has to

come from σ varying, which is why the schedule is periodic rather than merely high.

t(s)	σ /yr	band	cells	ladder
0	456%	\$1.239	14	9.7x ... 33.4x
24	714%	\$1.348	14	9.7x ... 33.4x
48	255%	\$0.539	14	9.7x ... 33.4x
72	143%	\$0.634	14	9.7x ... 33.4x

Same ladder throughout — the difficulty knob is band height, not payout.

VOLT is demo-only, and that is a security boundary rather than a preference. The browser generates the path, so the browser can read the seed and know the future. Harmless for play money; disqualifying for real. Promoting it would mean driving the path from a server-held hash chain revealed link by link — the shape Death Fun already uses.

6. Death Fun

A mines-style board. Safe tiles raise the multiplier; a skull ends the round; stopping banks it. Fair odds minus the same 7% edge:

$$\begin{aligned} P(\text{survive } n \text{ picks}) &= \prod (\text{safe} - i) / (\text{total} - i), \quad i = 0 \dots n-1 \\ \text{multiplier} &= (1 - 0.07) / P(\text{survive}) \end{aligned}$$

There is no external price to settle against here, so this is where commit-reveal is the **right** tool rather than a substitute for one.

Phase	What is published
before the first tap	<code>sha256(serverSeed)</code>
during the round	nothing — every unopened tile reads hidden
after the round	<code>serverSeed</code>

The board comes from `HMAC(serverSeed, clientSeed:nonce:i)`. Committing to the seed hash first means no board can be chosen after seeing where you click; the client supplying `clientSeed` means no unlucky board can have been prepared for a specific player in advance. Replaying the published seed reproduces the board exactly — both its shape and its skulls, since the shape is carved from the same stream.

The board is a random shape cut out of a `dim × dim` grid, so the tile count varies per round. Skull count is `round(density × tiles)`.

Difficulty	Grid	Density	First-tap safe
NORMAL	3×3	0.55	~45%
MEDIUM	6×6	0.62	~38%
HARD	13×13	0.72	~28%
ULTRA	20×20	0.84	~16%

7. Custody and settlement

Two constraints pull in opposite directions. Nobody plays a game that asks for a wallet signature on every bet. Nobody trusts a balance that lives in a company wallet. The split:

Layer	Where	Why
Custody	on-chain (<code>GameVault</code>)	funds must be auditable, not in a personal wallet
Play	off-chain (Postgres)	must be instant, gasless, signature-free

Settlement

external (exchange)

must not be ours to decide

```
connect wallet      no signature
  → get test tokens no signature
  → play            ledger settles instantly
  → deposit         approve + GameVault.deposit() ← the only user signature
  → withdraw        operator signs an EIP-712 voucher, backend relays
```

The client names a cell; it does not describe one

In real-money mode the browser sends only a settlement time and a price band. The server validates that band against **its own** lattice, prices it from **its own** market read, and debits the stake atomically. A multiplier the client reports is never used.

Settlement uses a price this server does not remember

`/api/game/tap/settle` resolves each due bet against the exchange's published 1-second bar for that instant, fetched at settlement time. Deliberately not a price we cached: a player can re-query the same public endpoint and check the result, and a serverless function has no memory worth trusting anyway, since instances come and go between a bet and its settlement. The price and volatility each bet was quoted on are stored, so any payout can be re-derived afterwards.

Voiding is not a free option

When the settlement price cannot be fetched, the bet is voided and the stake refunded rather than resolved on a guess. But the player triggers settlement, so cheap voiding would mean "bank the winners, retry the losers until the oracle blinks". Voiding therefore requires **6 failed attempts spread over at least 5 minutes, at least 10 minutes past due**, and bars the oracle has already seen are persisted.

Two keys, not one

`GameVault` separates two roles, and the constructor rejects a deployment where they are the same address :

Role	Signs	Lives
owner	rotates the operator	a wallet, never a server
operator	every withdrawal voucher	the server

The operator key is exposed by construction — the server has to sign with it on every cash-out. The owner key is what revokes it when that server is compromised. Collapse them and the revoke does not exist: whoever takes the hot key also takes the ability to rotate it. `setOperator` and `transferOwnership` enforce the same rule from the other directions.

— 8. Attack surface

A house that can be arbitrated is not a business. Adversarial review and playtesting found five ways a player could construct a winning request. All five are closed; four carry a regression test that reproduces the original exploit.

Exploit	Closed by
Client-shaped odds — draw your own band, solve for the cell the model prices worst	server snaps every band to its own lattice
Stale-quote arbitrage — our price ran a second behind the market	fresh quote per bet; the quote's age is charged to the model
Refunds as a free option — retry a losing bet until it voids	voiding needs repeated failures over real elapsed time
Cancelling after the fact — re-tap a resting bet to refund it	cancellation removed; a placed bet stands
A stalled market — the estimate breaks, a fallback kept quoting generously	fallback removed; no measurement, no quote

Each was a hole the bankroll would have drained through quietly in production.

9. What is verifiable, and what is still trusted

Stating the boundary is more useful than overstating the guarantee.

Verifiable by a third party

- Every Tap outcome, against the exchange's public 1-second bar
- Every Death Fun board, by replaying the published seed
- The exact house edge, by inverting the published payouts
- Both contracts' source, verified on Blockscout
- The estimator, the edge and the synthetic market, by running the suites

Still trusted

- **The ledger.** Play settles off-chain, so the server is authoritative for balances. **GameVault** removes external theft and makes deposits and withdrawals transparent; it does not make the operator trustless. Full trustlessness needs on-chain settlement or fraud proofs, which is out of scope for this version.
- **The quote.** The server prices from a market read it takes itself. It is bound afterwards — the horizon runs from the quote's own timestamp and the quote's age is charged to the model — but the read is ours.
- **VOLT's path.** Generated in the browser, hence demo-only.

10. Deployment status

Live on Giwa Sepolia (chainId 91342), source-verified on Blockscout:

Contract	Address
TestKRW (tKRW)	0x616cb26e3Af3895DEAc5A53f760ECEFaEF4e78bc
GameVault	0xc0c7A3DF600263492225d5dACecd5C036EF80B48

The public demo at voltaction.xyz is play-money only. Real-money mode — the server-authoritative ledger, and vault deposits and withdrawals — is written and tested, and the contracts above are deployed and verified, but it is switched off in that deployment: **NEXT_PUBLIC_REAL_MODE**, **NEXT_PUBLIC_GAMEVAULT_ADDRESS** and **OPERATOR_PRIVATE_KEY** are unset, so the REAL toggle renders disabled, and the vault's bankroll has never been funded. Opening the demo and finding real money unavailable is correct, not a fault.

What is unproven is demand.

Reproducing the claims

```
cd web && npm install
npm run verify:vol      # estimator recovers a known volatility; refuses when it cannot
npm run verify:edge     # Monte-Carlo of the realised edge across regimes
npm run verify:synth    # VOLT's edge is exact on every offered cell

cd ../contracts && npm install && npm test    # 13 tests
```

11. Parameters

Symbol	Value	Meaning
HOUSE_EDGE	0.07	edge, identical in both games
MAX_MULT	50	largest multiplier offered
BAND_SIGMA_FRACTION	0.2436	band height as a fraction of σ_{ref}
H_REF_SEC	28	horizon the band height is sized at
COL_INTERVAL_MS	3400	spacing between settlement columns
MIN/MAX_BET_HORIZON_SEC	10 / 60	how far ahead a bet may be placed
MAX_OPEN_BETS	60	open bets per account
VOL_WINDOW_SEC	600	slow estimation window
VOL_SHORT_SEC	90	fast window; pricing takes the larger
VOL_FINE/COARSE_SEC	1 / 5	the two sampling grids
VOL_MIN_SAMPLES	40	fine returns required before quoting
VOL_MIN_SNR	0.65	signal share required after noise subtraction
VOL_MIN / VOL_MAX	2×10^{-5} / 8×10^{-4}	guard rails; outside them, refuse
SIG_CALM / SIG_STORM	2.2×10^{-4} / 1.6×10^{-3}	VOLT's declared σ range (~124% / ~899% a year)
PERIOD_A / PERIOD_B	97 s / 31 s	VOLT's two schedule periods
SYNTH_STEP_MS	100	VOLT's generation lattice
VOID_MIN_ATTEMPTS	6	failed settlements required to void
VOID_MIN_AGE_MS	10 min	how far past due before voiding is possible
VOID_MIN_ATTEMPT_SPAN_MS	5 min	span those attempts must cover

Source: github.com/mycyi1994-hash/giwavolt · MIT